SOF 103
C AND C++ PROGRAMMING

Lecture 3
Operators and Control Structures

LAST WEEK REVIEW

- **Variables must be declared before use them in the program.**

Syntax: *data_type variable_name;*

- **Variable names in C++ can range from 1 to 255 characters.**
- **All variable names must begin with a letter of the alphabet or an underscore(_).**
- **After the first initial letter, variable names can also contain letters, underscore and numbers.**
- **Variable names are case sensitive.**
- **No spaces or special characters are allowed.**
- **You cannot use a C++ keyword (a reserved word) as a variable name.**

C++ DATA TYPES

There are 10 basic/primitive data types in C++.

Category	Data Type	Bytes
Integers	bool	1
	char	1
	int	4
	short	2
	long	4
	Unsigned int	4
	Unsigned short	2
	Unsigned long	4
Fractional numbers	float	4
	double	8

THE INPUT STATEMENT USING CIN OBJECT

- cin accepts data input from the console input like keyboard.
- cin takes the value from the user at run time and stores it in the variable.

The syntax:

```
cin >> variableName;
```

- Usually cin works together with the cout to make the program easy to understand during run time.

F = 9/5 * C + 32

- 9/5 = 1 **(instead of 1.8)**

→ **this will lead to wrong answer.**

To get the correct answer, use the following:

- 9.0/5 = 1.8
- 9/5.0 = 1.8
- (double) 9/5 = 1.8
- **F = (9 * C)/5 + 32**

CHAPTER OVERVIEW

1. Making decisions
2. Precedence of relational operators
3. if statement
4. if-else statement
5. switch statement
6. Logical operators
7. Formatting output
8. Loops / Iterations / Repetitions
   (for loop, while loop, do-while loop)

1. MAKING DECISIONS

- **Relational Operators**
  - **Relational operators allow you to compare numeric and character values and determine whether one is greater than, less than, equal to, or not equal to another.**

Relational Operators	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

EXAMPLE OF RELATIONAL OPERATORS

- **The result of comparing 2 values is either True or False.**

- **Example Relational Expressions (Assume x = 10 and y = 7)**

Expression	Value
x < y	false, because x is not less than y.
x > y	true, because x is greater than y.
x >= y	true, because x is greater than or equal to y.
x <= y	false, because x is not less than or equal to y.
y != x	true, because y is not equal to x.

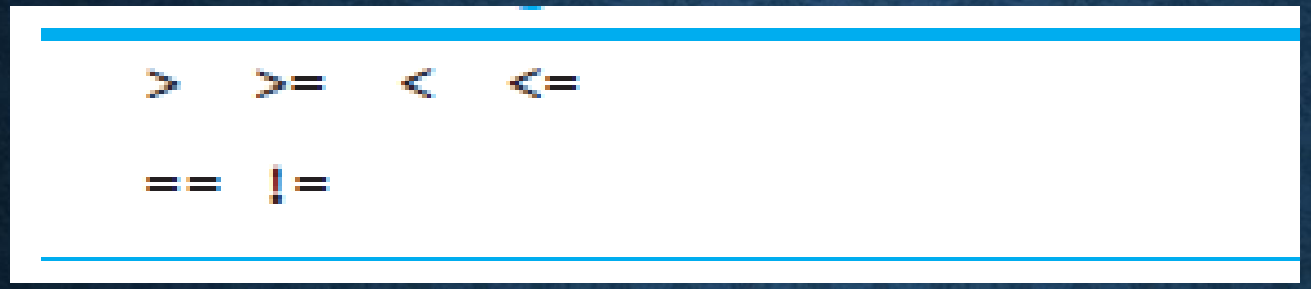
STATEMENTS USING RELATIONAL OPERATORS

- **Assume x = 10, y = 7, and z is an int or boolean**

Statement	Outcome
`z = x < y`	z is assigned 0 because x is not less than y.
`cout << (x > y);`	Displays 1 because x is greater than y.
`z = (x >= y);`	z is assigned 1 because x is greater than or equal to y.
`cout << (x <= y);`	Displays 0 because x is not less than or equal to y.
`z = (y != x);`	z is assigned 1 because y is not equal to x.
`cout << (x == (y + 3));`	Displays 1 because x is equal to y + 3.

2. PRECEDENCE OF RELATIONAL OPERATORS

- **Highest to Lowest**



- **Here is an example of how it is applied. If a = 9, b = 24, and c = 0, the following statement would print out 1**

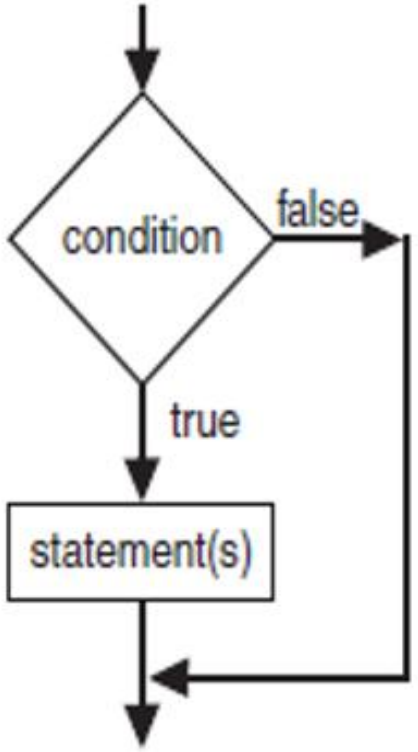
```
cout << (c == a > b);
```

- **Because of the relative precedence of the operators in this expression, a > b would be evaluated first.**

3. RECOMMENDED WAYS TO WRITE IF CODE

- **if statement is used to decide whether to execute a statement or to skip it from execution.**
- **if statement uses condition(s) to make the decision.**

```
if (condition)
{
        statement 1;
        statement 2;
            .
            .
        statement n;
}
```



CONDITION

- **Conditions are also called relational statements**
- **Conditions are expressions that results in true or false**
- **Conditions can be made use of relational operators**

Operator	Operator in C++	Description	Example
=	==	Equality	4 == 5 (false)
≠	!=	Not Equal	4 != 5 (true)
<	<	Less Than	4 < 5 (true)
>	>	Greater than	4 > 5 (false)
≤	<=	Less than or equal to	4 <= 5 (true)
≥	>=	Greater than or equell to	4 >= 5 (false)

EXAMPLE

- Write a program that asks user to enter a student's exam mark and displays "congratulations" to those whose marks are greater than or equal to 70.

```
#include<iostream>
using namespace std;
int main()
{
  int mark;
  cout << "Please enter a student mark: ";
  cin >> mark;
  if (mark >= 70)
      cout << "Congratulations";
}
```

Output 1

```
Please enter a student mark:  23
```

Output 2

```
Please enter a student mark:  85
Congratulations
```

4. IF/ELSE SELECTION STRUCTURE

- **Syntax of if/else statement:**

```
if ( condition ) {
        _____
        _____
        _____
}
else {
        _____
        _____
        _____
}
```

These statements are executed when the condition is true

These statements are executed when the condition is false

Note: True value is stored as “1” and false is stored as “0” in memory.

EXAMPLE

- Write a program that asks user to enter a positive integer N. Then the program should displays "EVEN" if N is an even number. Otherwise, display "ODD".

```
#include<iostream>
using namespace std;
int main( ) {
    int N;
    cout << "Please enter an integer: ";
    cin >> N;
    if (N%2 == 0)   // N is even
        cout << "N is EVEN" << endl;
    else
        cout << "N is ODD";
}
```

Output 1

```
Please enter an integer:  63
N is ODD
```

Output 2

```
Please enter a student mark:  78
N is EVEN
```

EXAMPLE

- Write a program to a take student mark from user. If mark is greater than 50 then display the message "You passed", otherwise display "You failed"

```
int main( )  {
        int mark;
        cout << "Please enter a student mark: ";
        cin >> mark;
        if (mark >= 50)
                cout << "You passed";
        else
                cout<<"You failed";
}
```

EXAMPLE: CALCULATE STUDENT GRADES

Write a program to take a student mark from user. The assign the grade to student based on the following table:

Mark	Grade
Mark >= 90	A
80 <= Mark < 90	B
70 <= Mark < 80	C
60 <= Mark < 70	D
Mark < 60	Fail

EXAMPLE: CALCULATE STUDENT GRADES

```
1. #include <iostream>
2. using namespace std;
3. int main ()
4. {
5.     float marks;
6.     cout << "Please enter the mark of a student:";
7.     cin >> mark;
8.     if(mark >= 90)
9.         cout << "A";
10.    else if(mark >= 80)
11.            cout << "B";
12.        else if(mark >= 70)
13.                Cout << "C";
14.            else if(mark >= 60)
15.                    Cout << "D";
16.                else
17.                    Cout << "Fail";
18.    return 0;
19. }
```

80 ≤ mark < 90

70 ≤ mark < 80

60 ≤ mark < 70

mark < 60

5. SWITCH STATEMENT

- **Switch statement** is used when we need to select single option between multiple choices
- Syntax:

```
1. #include <iostream>
2. using namespace std;
3. int main ()
4. {
5.    int expression;
6.    switch (expression)
7.    {
8.        case 1: statement(s); break;
9.        case 2: statement(s); break;
10.       case 3: statement(s); break;
11.       .
12.       .
13.       .
14.       case n: statement(s); break;
15.       default: statement(s);
16.   }
17.   return 0;
18. }
```

EXPLANATION

```
switch (expression)
{
    case 1: statement(s); break;
    case 2: statement(s); break;
    case 3: statement(s); break;
    .
    .
    .
    case n: statement(s); break;
    default: statement(s);
}
```

Break is used to stop the execution of other case statements

These values are used for equality with the expression and only corresponding statements will be executed

USING 'SWITCH' STATEMENT FOR MENUS

```
#include <iostream>
using namespace std;

int main()
{
    int choice;
    cout << "\n Choose 1 to see the next homework assignment.";
         << "\n Choose 2 for your grade on the last assignment.";
         << "\n Choose 3 for assignment hints.";
         << "\n Choose 4 to exit this program.";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice)  {
        case 1: // Code to display next assignment.
                break;
        case 2: // Code to display student grade.
                break;
        case 3: // Code to display assignment hints.
                break;
        case 4: cout << "End of Program.";
                break;
        default: cout << "Not a valid choice. Enter your choice again."
    }
    return 0;
}
```

6. LOGICAL OPERATORS

- **Logical operators connect two or more conditions (relational expressions) into one or reverse the logic of an ex**

Operator	Meaning	Effect
&&	AND	Connects two expressions into one. Both expressions must be true for the overall expression to be true.
||	OR	Connects two expressions into one. One or both expressions must be true for the overall expression to be true. It is only necessary for one to be true, and it does not matter which.
!	NOT	Reverses the "truth" of an expression. It makes a true expression false, and a false expression true.

LOGICAL AND OPERATOR (&&)

- **AND** operator is used when all the conditions in simple conditions must be true to evaluate the overall result into true
- The basic rule for **AND** operator (**&&**) are as below

Condition 1	Condition 2	Condition 1 && Condition 2
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

WRITE A PROGRAM THAT WILL GIVE 10 PERCENT RAISE TO THOSE EMPLOYEES WHOSE SALARY IS LESS THAN OR EQUAL TO $2000 AND GREATER THAN OR EQUAL TO $1200.

```
#include<iostream>
using namespace std;
int main( )
{
    int salary;
    cout << "Enter employee salary ";
    cin >> salary;
    if (salary  >= 1200 && salary <= 2000)
        salary = salary + (salary * 0.10);
    else
        cout << "No increase in salary ";
    cout << "New salary is = "<<salary;
}
```

LOGICAL OR OPERATOR (||)

- **OR** operator is used when at least one of the simple conditions need to be true to evaluate the overall result into true
- The basic rule for OR operator are as below

Condition 1	Condition 2	Condition 1 || Condition 2
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Write a program which increases 10% salary of the employees who are working in category 3 or whose salary is less than 5000

```
int main( )  {
    int salary, category;
    cout << "Enter employee salary ";
    cin >> salary;
    cout << "Enter employee category ";
    cin >> category;
    if (salary < 5000 || category == 3)
        salary = salary + salary * 0.10;
    else
        cout << "No increase in salary ";
    cout << "The salary is "<<salary;
}
```

7. FORMATTING OUTPUT

- **cout** provides ways to format data as it is being displayed. This affects the way data appears on the screen.

```
// This program displays three rows of numbers.
int main()
{
   int num1 = 2897, num2 = 5,    num3 = 837,
       num4 = 34,   num5 = 7,    num6 = 1623,
       num7 = 390,  num8 = 3456, num9 = 12;

   // Display the first row of numbers
   cout << num1 << "   " << num2 << "   " << num3 << endl;

   // Display the second row of numbers
   cout << num4 << "   " << num5 << "   " << num6 << endl;

   // Display the third row of numbers
   cout << num7 << "   " << num8 << "   " << num9 << endl;

   return 0;
}
```

PROGRAM OUTPUT

Program Output

```
2897   5   837
34   7   1623
390   3456   12
```

- The numbers do not line up in a column. Output looks messy.
- We can format the output by using **setw** manipulator.
- Here is an example of how to use it:




```
value = 23;
cout << setw(5) << value;
```

- The number '5' in the parenthesis specifies the field width. It means 23 will be displayed in a field of 5 spaces.

PROGRAM TO DEMONSTRATE SETW MANIPULATOR

```
// This program uses setw to display three rows of numbers so they align.
#include <iostream>
#include <iomanip>                    // Header file needed to use setw
int main()
{
   int num1 = 2897, num2 = 5,    num3 = 837,
       num4 = 34,   num5 = 7,    num6 = 1623,
       num7 = 390,  num8 = 3456, num9 = 12;

   // Display the first row of numbers
   cout << setw(6) << num1 << setw(6) << num2 << setw(6) << num3 << endl;

   // Display the second row of numbers
   cout << setw(6) << num4 << setw(6) << num5 << setw(6) << num6 << endl;

   // Display the third row of numbers
   cout << setw(6) << num7 << setw(6) << num8 << setw(6) << num9 << endl;

   return 0;
}
```

OUTPUT:

```
2897     5   837
  34     7  1623
 390  3456    12
```

8. LOOPS

Repetition structure

- It is used to repeat statement(s) to specified number of time

1. Deterministic Loop

- When the number of repetitions are known in advance

2. Non Deterministic Loop

- When number of repetitions are not known in advance

TYPES OF LOOP IN C++

- **for loop**
  - **Deterministic loop structure – used for known number of repetitions**
  - **It is repeated until some given condition become false**
- **while Loop**
  - **Non Deterministic loop structure – used for unknown number and known number of repetition.**
  - **It is also repeated until some specified condition becomes false**
- **do–while Loop**
  - **It is repeated until some given condition becomes false**
  - The difference between 'while and do-while' is that do while will execute at least once, even if the condition is false initially.

FOR STATEMENT

- The simplest loop structure

- Syntax

```
for (initialization; condition; increment)
{
        // Program statements
        body of loop;
}
```

EXAMPLE 1

- Write a program to display numbers from 1 to 10

```
#include <iostream>
using namespace std;
int main()
{
        int i;
        for (i = 1; i<=10; i++)
                cout << i << endl;
        cout << i << endl;
}
```

Output

```
1
2
3
4
5
7
8
9
10
11
```

EXAMPLE 2

- Write a program to display even numbers from 0 to 100

```
#include <iostream>
using namespace std;
int main()
{
        for (int i = 0; i<=100; i=i+2)
                cout << i << endl;
}
```

Output

```
0
2
4
6
8
10
. . .
. . .
. . .
```

EXAMPLE 3

- Write a program to display even numbers from 1 to 100

```
#include <iostream>
using namespace std;
int main()
{
        for (int i = 1; i<=100; i++)
        {
                if (i%2 == 0)
                    cout << i << " is even" << endl;
        }
}
```

EXAMPLE 4

- Write a program to add 5 integers entered by the user and print out the sum

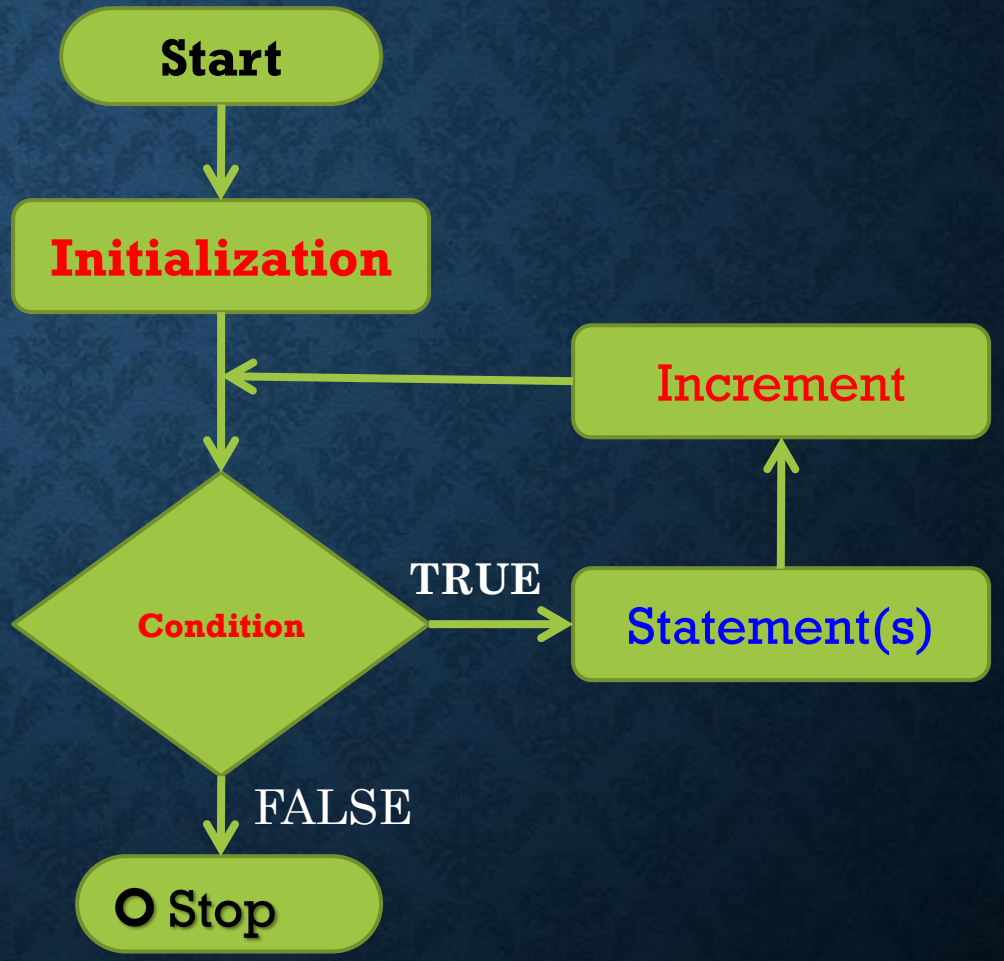
```
#include <iostream>
using namespace std;
int main( ) {
        int num, sum=0;
        for (int i = 1; i<=5; i++)
        {
            cout >> "Enter an integer: ";
            cin >> num;
            sum += num;
        }
        cout << "Sum = " << sum << endl;
        return 0;
}
```

```
Enter an integer: 10
Enter an integer: 8
Enter an integer: 7
Enter an integer: 15
Enter an integer: 20
Sum = 60
```

sum
10
18
25
40
60

WHILE LOOP

```
counter = start_value;
while (condition)
{
        Statement(s);
        Increment counter;
}
```



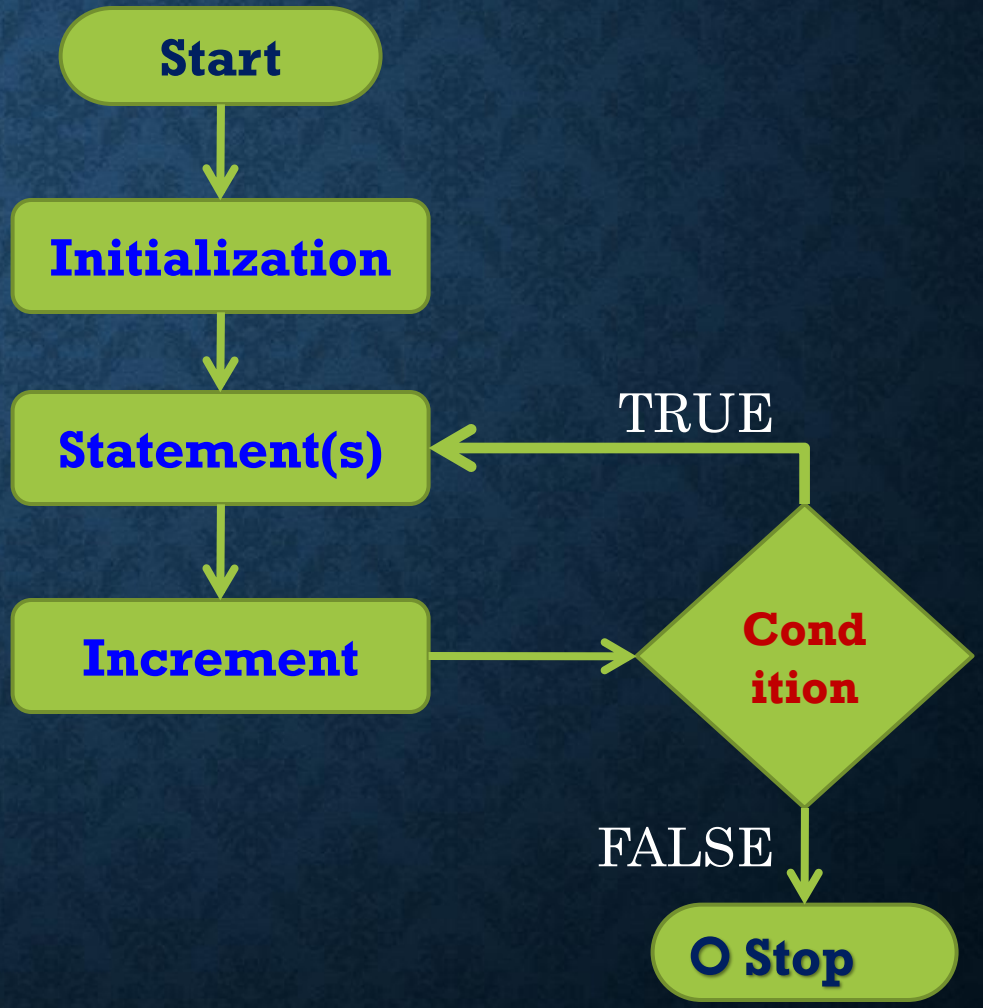
WHILE LOOP EXAMPLE

```
counter = 1;
while (counter <= 3)
{
      cout << counter;
      counter++;
}
cout << endl << counter;
```

counter	counter <= 3	output
1	T	1 2 3
2	T	4
3	T	
4	F	

DO–WHILE LOOP

```
counter = start_value;
do
{
    statements;
    Increment counter;
} while (counter <= 5);
```



DO–WHILE LOOP EXAMPLE

```
#include <iostream>
using namespace std;

int main()
{
    int  counter;
    counter = 10;
    do
    {
        cout << counter << endl;
        counter = counter - 1;
    } while (counter >= 1);
}
```

```
10
9
8
7
6
5
4
3
2
1

Process returned 0 (0x0)   execution time : 0.063 s
Press any key to continue.
```

NON DETERMINISTIC LOOPS: WHILE LOOP

Write a program that will add a list of numbers entered by the user

```
1. #include <iostream>
2. using namespace std;
3. int main ()
4. {
5.    int sum = 0, num;
6.    char ch;
7.    cout << "Enter number to add ";
8.    cin >> num;
9.    sum = sum + num;
10.   cout << "do you want to add next number? (y or n)";
11.   cin >> ch;
12.   while (ch == 'y')
13.   {
14.      cout << "Enter number to add ";
15.      Cin >> num;
16.      sum = sum + num;
17.      cout << "do you want to add next number? (y or n)";
18.      cin >> ch;
19.   }
20.   cout<<"The sum is = "<<sum;
21.   return 0;
22. }
```

NON DETERMINISTIC LOOPS: DO-WHILE LOOP

Write a program that will add a list of numbers entered by the user

```
1. #include <iostream>
2. using namespace std;
3. int main ()
4. {
5.     int sum = 0, num;
6.     char ch;
7.     do
8.     {
9.          cout << "Enter number to add ";
10.         cin >> num;
11.         sum = sum + num;
12.         cout << "do you want to add next number? (y or n)";
13.         cin >> ch;
14.     } while (ch == 'y');
15.
16.   cout << "The sum is = " << sum;
17.   return 0;
18. }
```

THE 'BREAK' STATEMENT

The **break** statement is used to escape early from a loop or to skip the remainder of a switch structure.

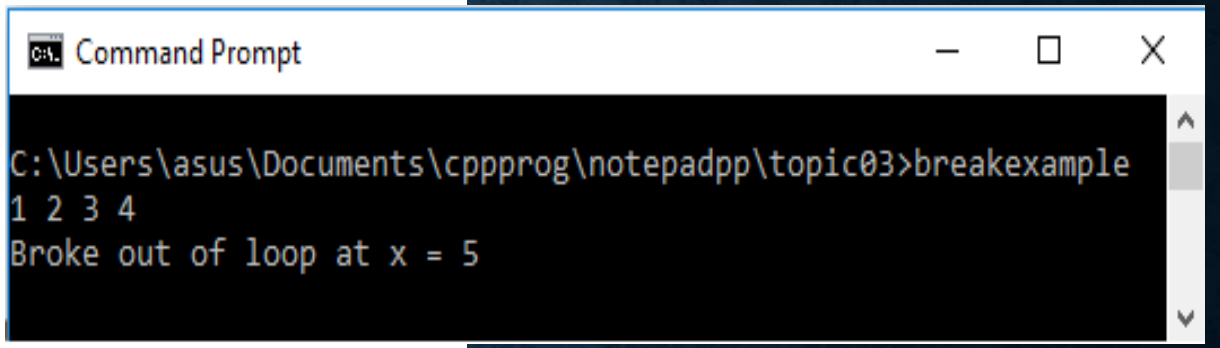
```
// Using break statement in a for loop.
#include <iostream>

using namespace std;

int main()
{
    int x; // declare an integer variable

    for(x = 1; x <=10; x++) {
        if(x == 5)
            break; // break out from for loop if x is 5.

        cout << x << " ";
    }
    cout <<"\nBroke out of loop at x = " << x << endl;
    return 0;
}
```



MORE ON 'BREAK' STATEMENT

```
1   //Sums a list of ten negative numbers.
2   #include <iostream>
3   using namespace std;
4
5   int main( )
6   {
7       int number, sum = 0, count = 0;
8       cout << "Enter 10 negative numbers:\n";
9
10      while (++count <= 10)
11      {
12          cin >> number;
13
14          if (number >= 0)
15          {
16              cout << "ERROR: positive number"
17                   << " or zero was entered as the\n"
18                   << count << "th number! Input ends "
19                   << "with the " << count << "th number.\n"
20                   << count << "th number was not added in.\n";
21              break;
22          }
23
24          sum = sum + number;
25      }
26
27      cout << sum << " is the sum of the first "
28           << (count - 1) << " numbers.\n";
29
30      return 0;
31  }
```

THE 'BREAK' STATEMENT IN NESTED LOOPS

- A *break* statement ends only the innermost loop that contains it.
- If you have a loop within a loop and a *break* statement in the inner loop, then the *break* statement will end only the inner loop.

```
for (int i=0; i < 10; i++)
{
      sum += i;
      for (int k=0; k < 10; k++)
      {
            product *= k;
            if (k == 5)
                  break;
      }
      cout << "Product = " << product << endl;
}
cout << "Sum = " << sum << endl;
```

THE 'CONTINUE' STATEMENT

- The **continue** statement, when executed in any loop structure, skips the remaining statements in the body of that structure, and proceeds with the next iteration of the loop.

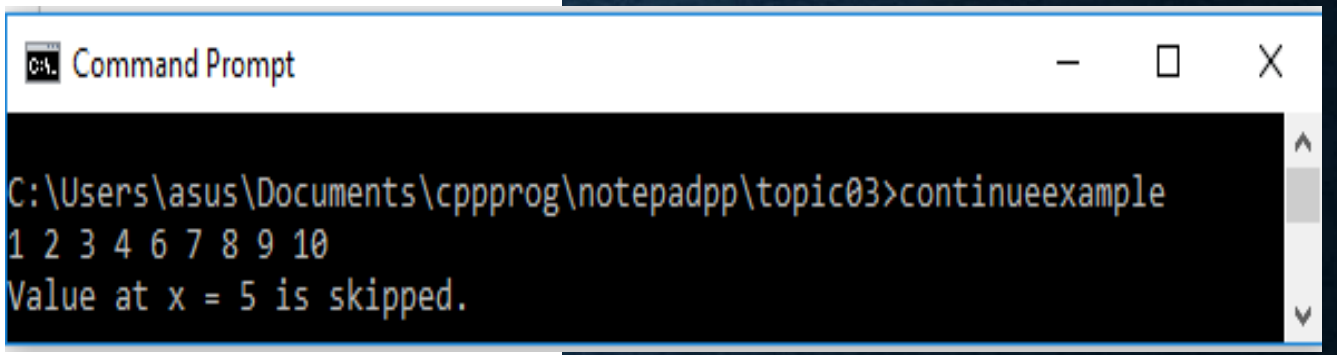
```
// Using continue statement in a for loop.
#include <iostream>

using namespace std;

int main()
{
    int x; // declare an integer variable

    for(x = 1; x <=10; x++) {
        if(x == 5)
            continue; // skip remaining code in loop only when x is 5.

        cout << x << " ";
    }
    cout <<"\nValue at x = 5 is skipped." << endl;
    return 0;
}
```



MORE EXAMPLE

- **break** Inside Nested Loops

```
#include <iostream>
using namespace std;

int main()  {
int weeks = 3, days_in_week = 7;

   for (int i = 1; i <= weeks; ++i) {
      cout << "Week: " << i << endl;

      for (int j = 1; j <= days_in_week; ++j) {
         // break during the 2nd week
         if (i == 2) {
            break;
         }
         cout << "    Day:" << j << endl;
      }
   }
}
```

MORE EXAMPLE

- **continue** Inside Nested Loops

```
#include <iostream>
using namespace std;

int main()  {
   int weeks = 3, days_in_week = 7;

   for (int i = 1; i <= weeks; ++i)  {
      cout << "Week: " << i << endl;

      for (int j = 1; j <= days_in_week; ++j)  {
         // continue if the day is an odd number
         if (j % 2 != 0)  {
            continue;
         }
         cout << "    Day:" << j << endl;
      }
   }
}
```

MORE EXAMPLE

- Use a while-loop to ask user to enter a series of integers. The loop will stop when user enters the sentinel value 999.

```
#include <iostream>
using namespace std;

int main()  {
   int num, sum=0;

   cout << "Enter an integer (999 to exit): ";
   cin >> num;
   while (num != 999)
   {
       sum += num;
       cout << "Enter an integer (999 to exit): ";
       cin >> num;
    }
}
```

```
#include <iostream>
using namespace std;

int main()  {
   int num, sum=0;

   while (true)
   {
      cout << "Enter an integer (999 to exit): ";
      cin >> num;

      if (num == 999)
          break;

       sum += num;
    }
}
```

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

REFERENCES

1. Problem Solving with C++ (10th Edition), Walter Savitch, Addison Wesley, 2018
2. CS201 Introduction to Programming, Virtual University of Pakistan
3. Paul J. Dietel, C++ How to Program (10th Edition), Pearson, 2017.
4. Turbo C programming for the PC, Robert Lafore

PROGRAMMING EXERCISE

- Write a program that will first display the menu below.

```
1  Addition
2  Subtraction
3  Multiplication
4  Division
5  Exit
```

- Then the program will prompt the user to enter 2 integers.
- After that the prompt the user to select the option from the menu to perform calculation on the 2 entered numbers.
- Lastly, the program should display the correct answer based on the user selection.